

Software Design Document (SDD)

Module	SPI Driver
Version	1.0.0
Team	Vania Leal · Carlos Cayetano · Carlos Araiza · Kheara Kieley
Date	29/05/2026
Institution	CETYS Universidad

1. Introduction

1.1 Purpose

This document describes the software design of the SPI driver module for the STM32F411RE microcontroller. The design is based on the requirements stated in the Software Requirements Specification (SRS) for the SPI driver. The document specifies data structures, register-level algorithms, and control flow so that an engineer can implement, integrate, and test the module without consulting the hardware manual for routine operations.

1.2 Scope

The SPI driver is a low-level software component that configures the SPI1 peripheral of the STM32F411RE and exposes a clean C API for higher-level modules. The driver operates in master, full-duplex, 8-bit mode with Mode 3 clock polarity (CPOL=1, CPHA=1) and software chip-select control. It has been validated with the ADXL345 3-axis digital accelerometer. No HAL, RTOS, or standard library dependencies exist within the driver itself.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SPI	Serial Peripheral Interface – a synchronous, full-duplex serial bus protocol
MOSI	Master Out Slave In – data line from master to slave
MISO	Master In Slave Out – data line from slave to master
SCK	Serial Clock – clock signal generated by the master
CS	Chip Select – active-low signal that selects a specific slave device
CPOL	Clock Polarity – defines the idle state of SCK (0=LOW, 1=HIGH)
CPHA	Clock Phase – defines the clock edge on which data is sampled
Mode 3	SPI clock mode with CPOL=1, CPHA=1; SCK idle HIGH, data captured on falling edge
SSM	Software Slave Management – bit in SPI_CR1 that enables GPIO-controlled CS
SSI	Internal Slave Select – bit in SPI_CR1; must be 1 when SSM=1 to avoid mode fault
DFF	Data Frame Format – bit in SPI_CR1; 0=8-bit, 1=16-bit
TXE	Transmit buffer Empty flag in SPI_SR; set when DR is ready for new data
RXNE	Receive buffer Not Empty flag in SPI_SR; set when DR contains received data

DR	Data Register of SPI1; shared register for TX and RX data
CR1	Control Register 1 of SPI1; configures mode, clock, frame format
CR2	Control Register 2 of SPI1; configures interrupts and DMA
SR	Status Register of SPI1; holds TXE, RXNE, BSY, and error flags
APB2	Advanced Peripheral Bus 2 – SPI1 clock source on STM32F4
AHB1	Advanced High-performance Bus 1 – GPIOA clock source on STM32F4
AF	Alternate Function – GPIO pin mode that routes the pin to a peripheral
ADXL345	3-axis digital accelerometer by Analog Devices; supports SPI and I2C
HSI	High-Speed Internal oscillator – 16 MHz default clock source on STM32F411RE
CMSIS	Cortex Microcontroller Software Interface Standard – register access macros
SRS	Software Requirements Specification
SDD	Software Design Document
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
NULL	A null pointer constant; used to indicate an absent or invalid pointer

1.4 Requirements

All functional and non-functional requirements for this module are specified in the SRS-SPI_Driver document located at: ../../SRS/SPI/SRS-SPI_Driver.md. The table below maps each SRS functional requirement to the driver function that implements it.

Requirement ID	Description	Implementing Function(s)
FR-1	The driver shall configure SPI1 to its default operating state.	spi_init()

FR-2	The driver shall enable clocking for the SPI1 hardware and associated GPIO.	<code>spi_init()</code>
FR-3	The driver shall transmit a sequence of bytes from a caller-supplied buffer.	<code>spi_transmit()</code>
FR-4	The driver shall synchronize each transmitted byte with the SPI clock via TXE polling.	<code>spi_transferByte()</code>
FR-5	The driver shall receive a sequence of bytes into a caller-supplied buffer.	<code>spi_receive()</code>
FR-6	The driver shall synchronize each received byte with the SPI clock via RXNE polling.	<code>spi_transferByte()</code>
FR-7	The driver shall assert the CS line LOW to select the target device.	<code>spi_csEnable()</code>
FR-8	The driver shall de-assert the CS line HIGH to release the target device.	<code>spi_csDisable()</code>

2. System Architecture

2.1 High-Level Design

The SPI driver sits within a four-tier layered architecture. The Application Layer (`main.c` and `sensor.c`) calls the SPI driver API to communicate with external devices such as the ADXL345. The SPI Driver Layer owns all SPI1 register interactions and exposes the six public functions defined in `spi.h`. The GPIO Driver Layer is used by `spi_init()` to configure pin modes and alternate functions. The Hardware Layer comprises the SPI1, GPIOA, and RCC peripheral registers mapped by CMSIS.

The separation of concerns ensures that changes to the pin assignment or clock configuration remain confined to `spi_init()`, while `spi_transmit()`, `spi_receive()`, and `spi_transferByte()` depend only on the SPI1 data path registers (SR and DR).

2.2 Module Interfaces

The public API is declared in `spi.h`. The implementation file `spi.c` includes `spi.h` and depends on the following CMSIS-provided definitions:

- SPI1, SPI_CR1_SSM, SPI_CR1_SSI, SPI_CR1_MSTR, SPI_CR1_CPOL, SPI_CR1_CPHA, SPI_CR1_BR_Pos, SPI_CR1_SPE
- SPI_SR_TXE, SPI_SR_RXNE
- RCC, RCC_APB2ENR_SPI1EN
- GPIOA, GPIO_PIN_4 through GPIO_PIN_7 (via `gpio_driver.h`)
- `__IO` – volatile qualifier macro from `core_cm4.h`

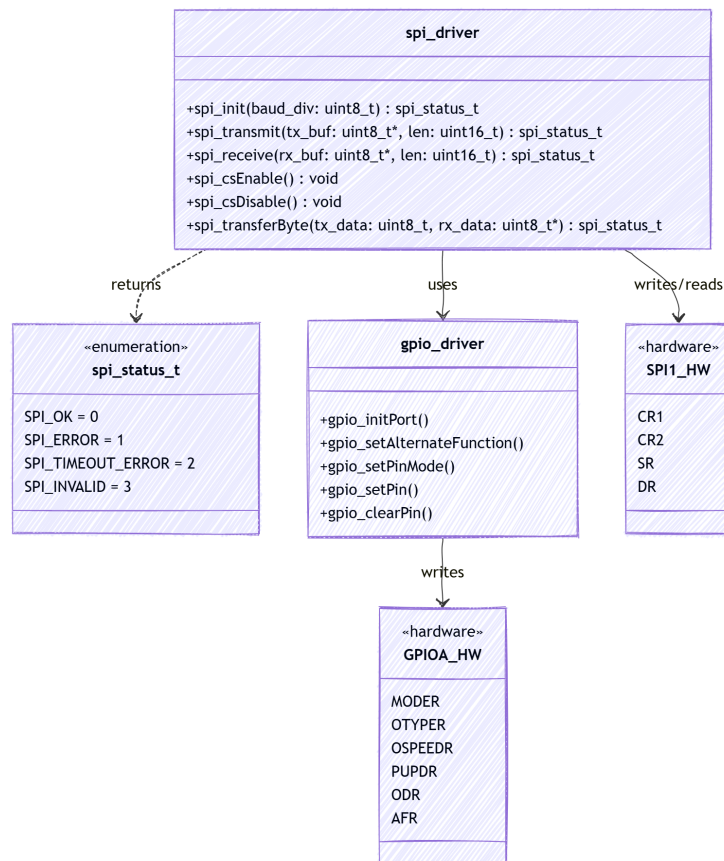
The driver depends on one peer module: `gpio_driver` (`gpio_initPort`, `gpio_setAlternateFunction`, `gpio_setPinMode`, `gpio_setPin`, `gpio_clearPin`). No other drivers are required.

3. Detailed Design

3.1 Class Diagram

Figure 1 presents the static structure of the SPI driver. The spi_driver class groups the six public operations. The spi_status_t enumeration defines the four return codes shared by all non-void functions. The driver delegates GPIO configuration to the gpio_driver module and directly accesses SPI1 and GPIOA hardware registers.

[Mermaid Diagram – Figure 1 – Class Diagram]

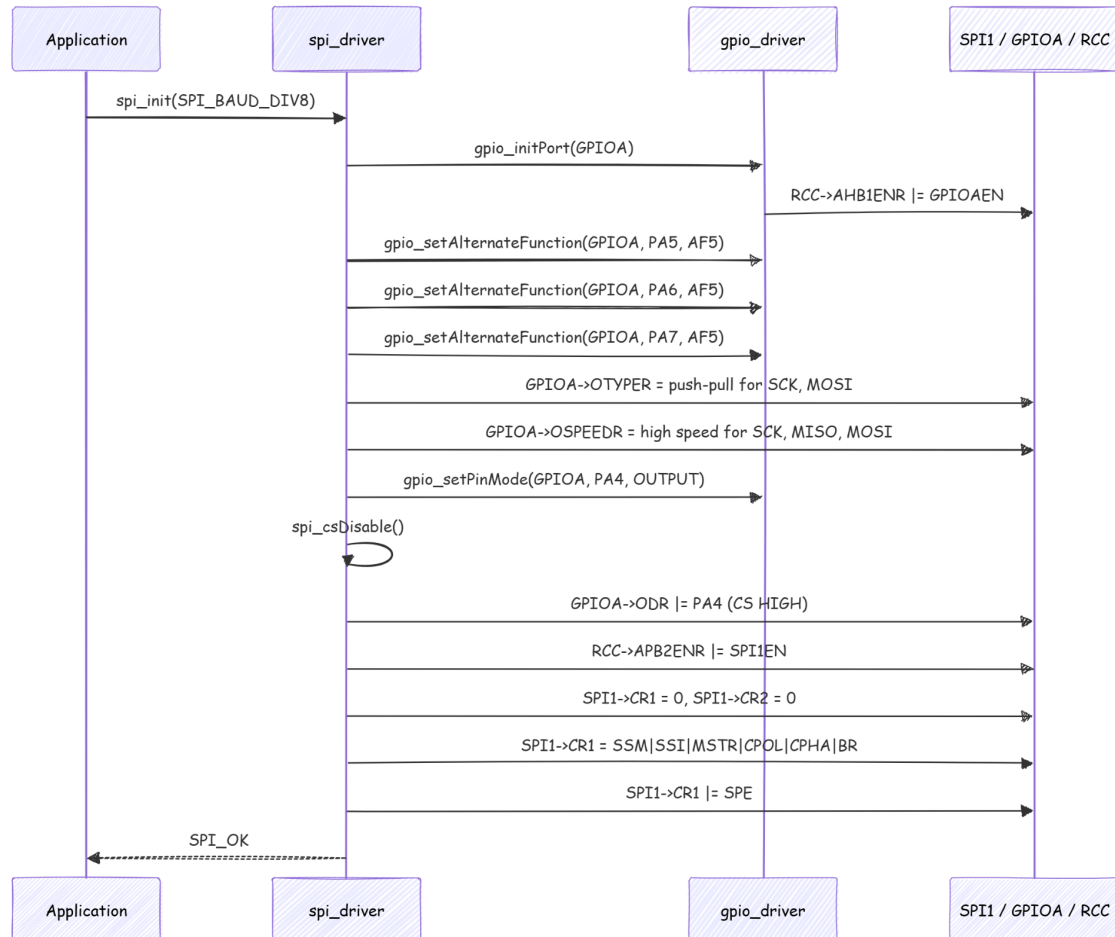


3.2 Module Behavior

3.2.1 spi_init – Sequence Diagram

Figure 2 shows the collaboration between the application, the SPI driver, the GPIO driver, and the hardware registers during initialization. The driver enables peripheral clocks, configures GPIO pins, sets SPI1 into Mode 3 master operation, and ends by asserting the SPE bit.

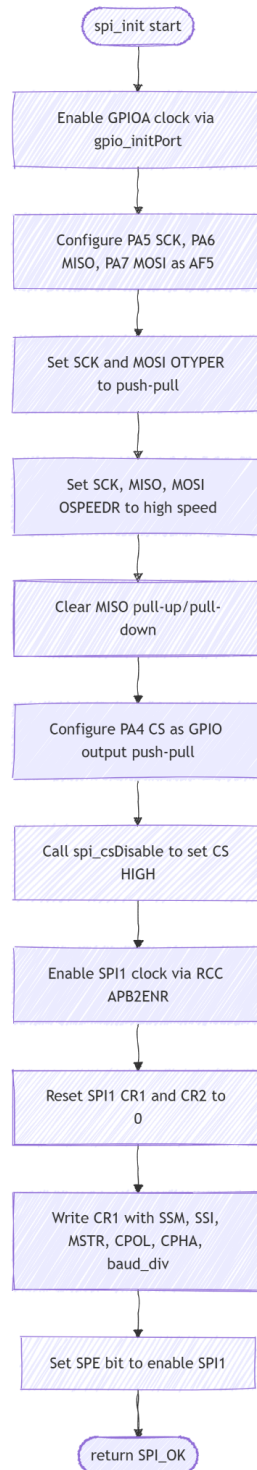
[Mermaid Diagram – Figure 2 – Sequence Diagram: spi_init]



3.2.2 spi_init – Activity Diagram

Figure 3 traces the step-by-step execution path of spi_init(). The function always returns SPI_OK; error states within initialization are not propagated because the peripheral either configures correctly or the hardware is absent (a fatal condition outside the driver scope).

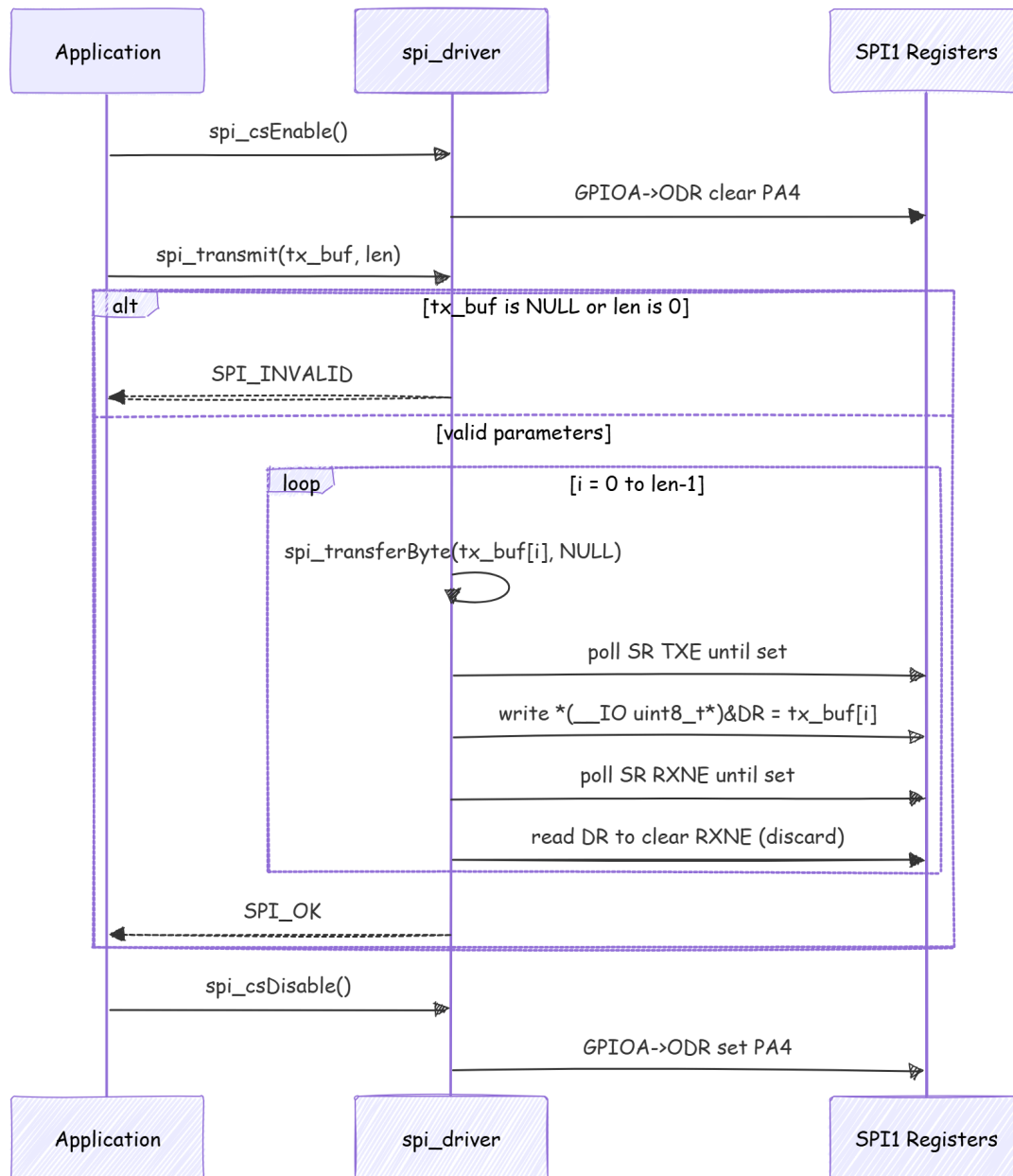
[Mermaid Diagram – Figure 3 – Activity Diagram: spi_init]



3.2.3 spi_transmit – Sequence Diagram

Figure 4 shows the transmit flow including CS management. The application is responsible for asserting and de-asserting CS before and after calling spi_transmit(). For each byte, spi_transferByte() polls TXE, writes the data register using an 8-bit cast, polls RXNE, and discards the received byte.

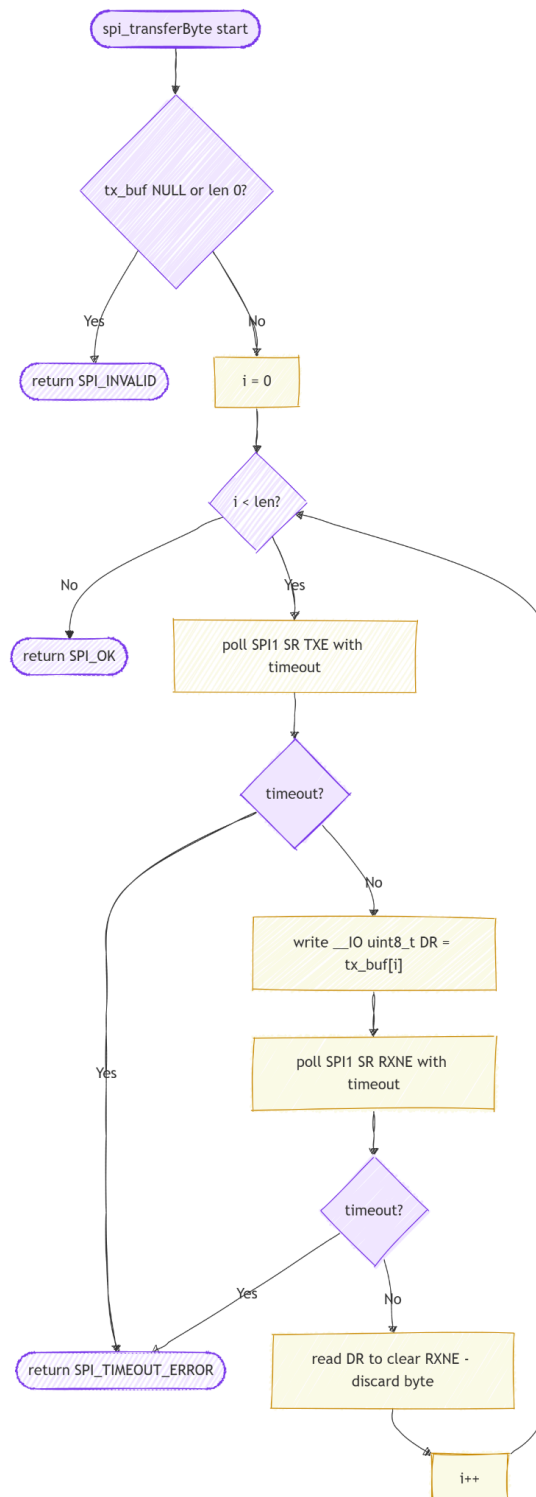
[Mermaid Diagram – Figure 4 – Sequence Diagram: spi_transmit]



3.2.4 spi_transferByte – Activity Diagram

Figure 5 details the core byte-transfer primitive used by both spi_transmit() and spi_receive(). The 8-bit pointer cast `(*(__IO uint8_t *)&SPI1->DR)` prevents compiler-generated 16-bit bus accesses that would corrupt the SPI frame. Both TXE and RXNE polling loops are guarded by a countdown timeout of SPI_TIMEOUT (100 000) cycles, after which SPI_TIMEOUT_ERROR is returned.

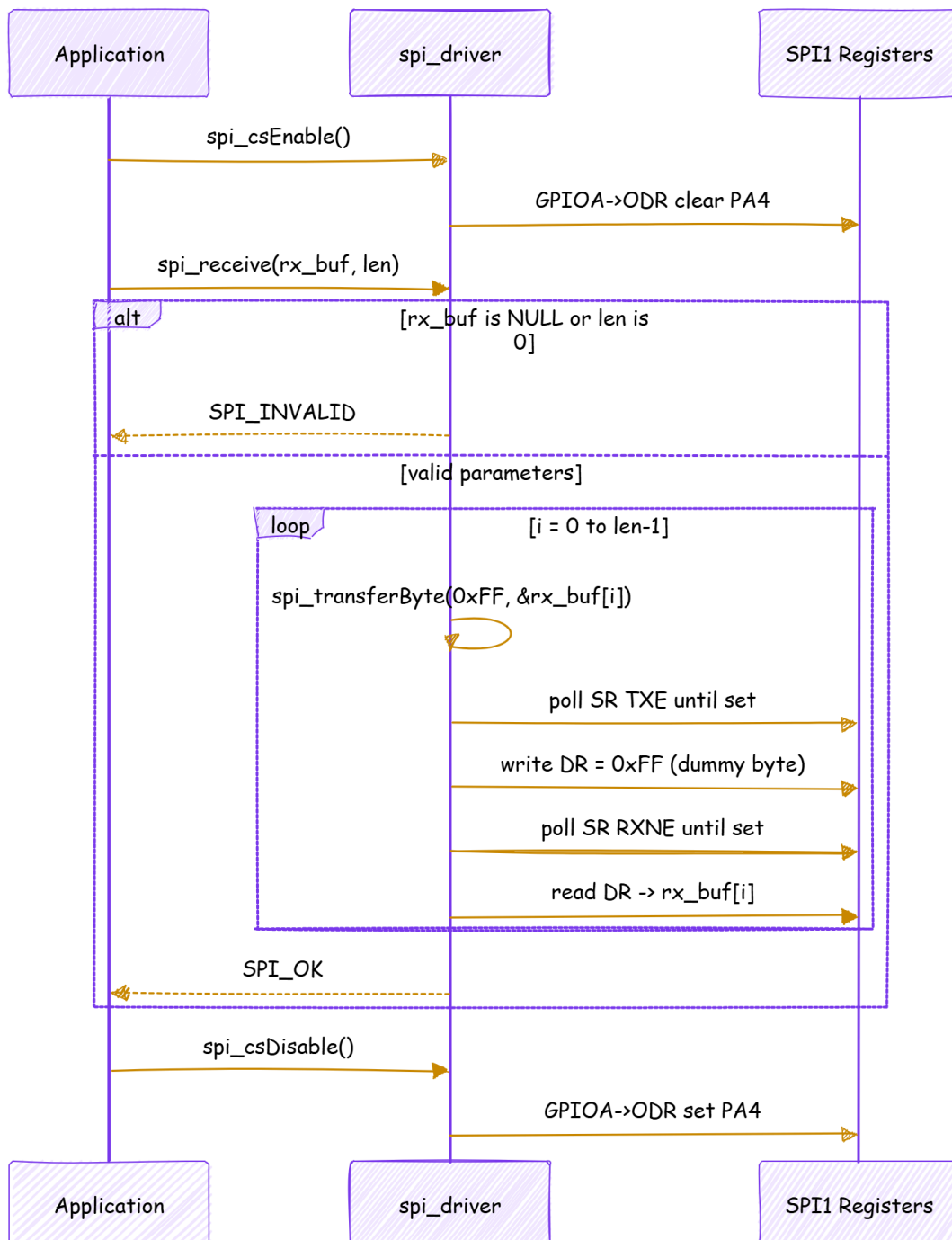
[Mermaid Diagram – Figure 5 – Activity Diagram: spi_transferByte (transmit path)]



3.2.5 spi_receive – Sequence Diagram

Figure 6 illustrates the receive operation. SPI is inherently full-duplex; to read data from a slave the master must clock out something. The driver sends dummy byte 0xFF for each byte to be received, which generates the required SCK pulses without corrupting the slave's state.

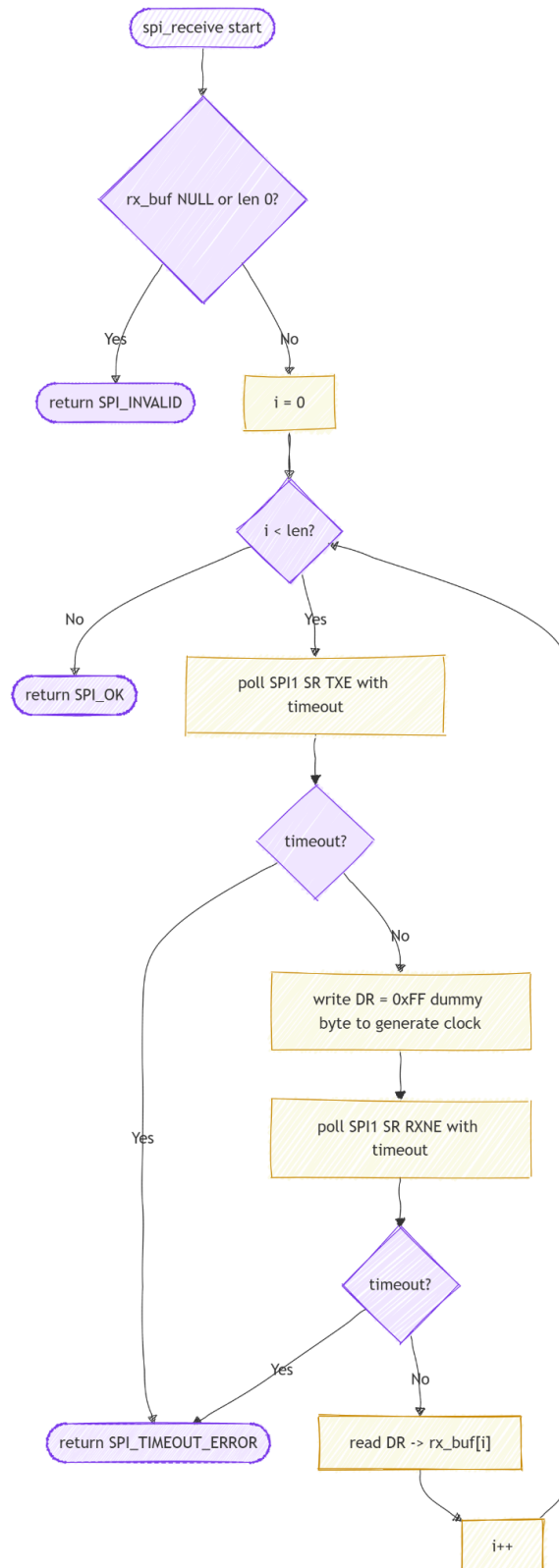
[Mermaid Diagram – Figure 6 – Sequence Diagram: spi_receive]



3.2.6 spi_receive – Activity Diagram

Figure 7 mirrors Figure 5 but highlights the dummy-byte mechanism: the same spi_transferByte() primitive is called with tx_data=0xFF and a valid rx_data pointer. Every iteration writes 0xFF to DR and captures the slave response into rx_buf[i].

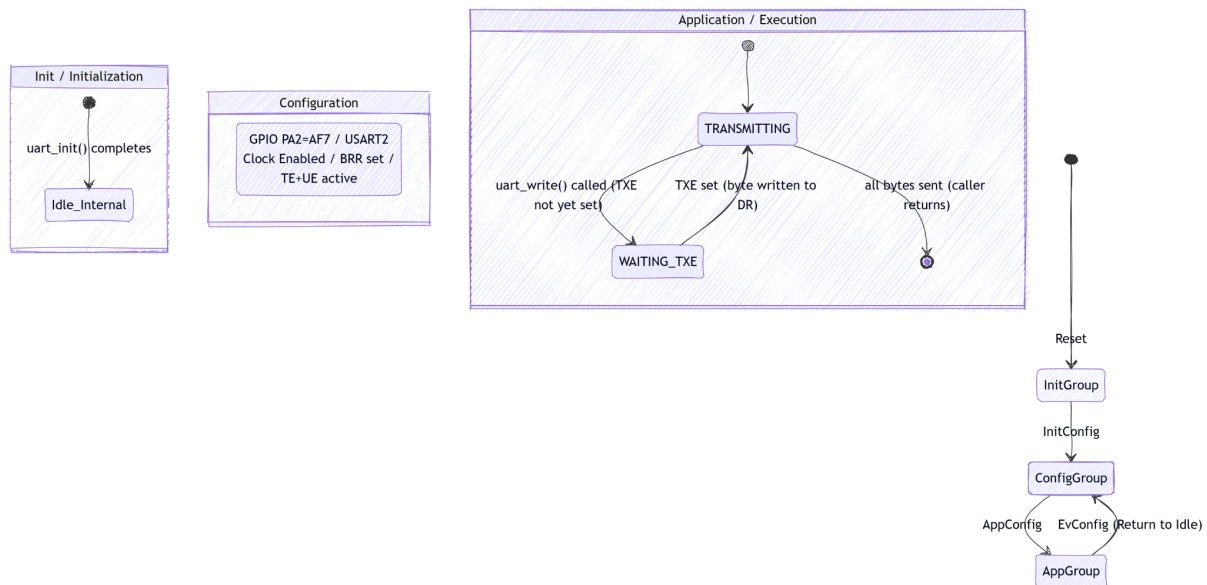
[Mermaid Diagram – Figure 7 – Activity Diagram: spi_receive (receive path)]



3.2.7 State Machine

Figure 8 shows the finite-state machine of the SPI driver. The driver starts in the Uninitialized state after a power-on reset. A successful call to `spi_init()` moves it to Idle. The CS_Active state is entered and exited via `spi_csEnable()` and `spi_csDisable()`. Data transfer states (Transmitting, Receiving, TransferByte) are transient; they return to CS_Active on success or move to Error on timeout. The application recovers from Error by calling `spi_csDisable()` or re-initializing the peripheral.

[Mermaid Diagram – Figure 8 – State Diagram: SPI Driver]



3.3 SPI Driver Description

The SPI driver abstracts the STM32F411RE SPI1 peripheral behind a six-function API. Initialization (`spi_init`) is a one-time configuration that enables clocks, maps GPIO pins to AF5, sets Mode 3 clock polarity ($CPOL=1$, $CPHA=1$), configures software chip-select, and enables the peripheral. After initialization the driver is stateless with respect to internal variables; its only state is held in hardware registers.

Data transfer relies on `spi_transferByte()`, a synchronous full-duplex primitive. Because SPI shifts one bit in and one bit out simultaneously per clock edge, the function must both write and read the data register in every invocation. Writing only – without reading – would cause the RXNE overrun flag to set on the next transfer, corrupting subsequent data. The DR register is accessed through an 8-bit volatile pointer cast `(*(__IO uint8_t *)&SPI1->DR)` to prevent GCC from widening the access to 16 bits, which would alter the DFF field behavior on some compiler versions.

CS management is deliberately external to `spi_transmit()` and `spi_receive()`. Multi-byte burst transactions on devices such as the ADXL345 require CS to remain asserted across all bytes. Embedding CS toggling inside the transfer functions would

prevent burst operations. The application or sensor driver is responsible for calling `spi_csEnable()` before and `spi_csDisable()` after each complete transaction.

4. Application Programming Interfaces

4.1 Data Types and Macros

`spi_status_t`: This enumeration is the return type of all non-void SPI driver functions. It encodes the outcome of each operation.

Enumerator	Value	Description
SPI_OK	0	Operation completed successfully.
SPI_ERROR	1	Generic unrecoverable error.
SPI_TIMEOUT_ERROR	2	A polling loop exceeded SPI_TIMEOUT iterations.
SPI_INVALID	3	A required pointer parameter is NULL or length is 0.

`SPI_BAUD_DIVx` macros: These eight macros (`SPI_BAUD_DIV2` through `SPI_BAUD_DIV256`, values 0–7) map to the `BR[2:0]` field in `SPI1_CR1`. With APB2 running at the default 16 MHz HSI frequency, the resulting SCK frequencies are 8 MHz, 4 MHz, 2 MHz, 1 MHz, 500 kHz, 250 kHz, 125 kHz, and 62.5 kHz respectively. `SPI_BAUD_DIV8` (2 MHz) is the recommended setting for ADXL345.

`SPI_TIMEOUT`: A `uint32_t` countdown constant set to 100 000. It guards every TXE and RXNE polling loop. If the counter reaches zero the loop exits and `SPI_TIMEOUT_ERROR` is returned, preventing the caller from blocking indefinitely on a stalled bus.

4.2 Error Codes

All non-void SPI driver functions return `spi_status_t`. The table below summarizes each code.

Code	Value	Meaning
SPI_OK	0x00	The operation succeeded.
SPI_ERROR	0x01	An unspecified hardware error occurred.
SPI_TIMEOUT_ERROR	0x02	A polling loop timed out waiting for TXE or RXNE.
SPI_INVALID	0x03	A NULL pointer or zero-length argument was passed.

4.3 Function Definitions

`spi_init`

Function Name	spi_init
Syntax	spi_status_t spi_init(uint8_t baud_div)
Parameters (in)	baud_div — Clock prescaler selector. Must be one of SPI_BAUD_DIV2 through SPI_BAUD_DIV256 (values 0–7). Only the lower 3 bits are used.
Parameters (inout)	— — —
Parameters (out)	— — —
Return value	spi_status_t — Always returns SPI_OK. The function does not report partial failure.
Description	Enables GPIOA and SPI1 clocks, configures PA5/PA6/PA7 as AF5 (SCK/MISO/MOSI), configures PA4 as GPIO output for CS (idle HIGH), sets SPI1 to master full-duplex 8-bit Mode 3 with software CS, applies baud_div, and enables the peripheral.
Constraints	The function always returns SPI_OK. If the GPIO or SPI hardware is absent or faulted, the failure is silent.
Available via	spi.h

spi_transmit

Function Name	spi_transmit
Syntax	spi_status_t spi_transmit(const uint8_t *tx_buf, uint16_t len)
Parameters (in)	tx_buf, len — tx_buf: pointer to the byte array to transmit (must not be NULL). len: number of bytes to send (must be > 0).
Parameters (inout)	— — —
Parameters (out)	— — —
Return value	spi_status_t — SPI_OK on success. SPI_INVALID if tx_buf is NULL or len is 0. SPI_TIMEOUT_ERROR if the bus stalls.
Description	Sends len bytes from tx_buf over SPI1. For each byte spi_transferByte() is called with a NULL rx_data pointer, discarding the simultaneously received byte. CS must be managed by the caller.
Constraints	Does not assert or de-assert CS. The caller must call spi_csEnable() before and spi_csDisable() after this function.
Available via	spi.h

spi_receive

Function Name	spi_receive
Syntax	spi_status_t spi_receive(uint8_t *rx_buf, uint16_t len)

Parameters (in)	len — Number of bytes to receive. Must be > 0.
Parameters (inout)	— — —
Parameters (out)	rx_buf — Buffer where received bytes are stored. Must not be NULL.
Return value	spi_status_t — SPI_OK on success. SPI_INVALID if rx_buf is NULL or len is 0. SPI_TIMEOUT_ERROR if the bus stalls.
Description	Reads len bytes into rx_buf by transmitting the dummy value 0xFF for each byte position. This generates the SCK pulses required for the slave to drive MISO. CS must be managed by the caller.
Constraints	Does not assert or de-assert CS. Caller is responsible for CS control around this call.
Available via	spi.h

spi_csEnable

Function Name	spi_csEnable
Syntax	void spi_csEnable(void)
Parameters (in)	— — —
Parameters (inout)	— — —
Parameters (out)	— — —
Return value	void — No return value.
Description	Drives PA4 LOW by calling gpio_clearPin(GPIOA, SPI_PIN_CS), selecting the target SPI device. The application must call this function before initiating any SPI transaction.
Constraints	Only one SPI device is supported on this CS line. Multi-device support requires additional GPIO pins managed externally.
Available via	spi.h

spi_csDisable

Function Name	spi_csDisable
Syntax	void spi_csDisable(void)
Parameters (in)	— — —
Parameters (inout)	— — —
Parameters (out)	— — —
Return value	void — No return value.

Description	Drives PA4 HIGH by calling <code>gpio_setPin(GPIOA, SPI_PIN_CS)</code> , deselecting the target SPI device. The application must call this function after each complete SPI transaction.
Constraints	Has no effect if CS is already HIGH.
Available via	<code>spi.h</code>

spi_transferByte

Function Name	<code>spi_transferByte</code>
Syntax	<code>spi_status_t spi_transferByte(uint8_t tx_data, uint8_t *rx_data)</code>
Parameters (in)	<code>tx_data</code> — Byte to transmit. Use <code>0xFF</code> when the caller only needs to receive.
Parameters (inout)	— — —
Parameters (out)	<code>rx_data</code> — Pointer to store the received byte. If <code>NULL</code> , the received byte is discarded.
Return value	<code>spi_status_t</code> — <code>SPI_OK</code> on success. <code>SPI_TIMEOUT_ERROR</code> if <code>TXE</code> or <code>RXNE</code> does not set within <code>SPI_TIMEOUT</code> cycles.
Description	Core full-duplex byte primitive. Polls <code>TXE</code> , writes <code>tx_data</code> to <code>DR</code> via an 8-bit pointer cast, polls <code>RXNE</code> , and reads <code>DR</code> . Used internally by <code>spi_transmit()</code> and <code>spi_receive()</code> , and also available directly for low-level sensor register access.
Constraints	The 8-bit <code>DR</code> pointer cast <code>((__IO uint8_t *)&SPI1->DR)</code> is required to prevent GCC from performing a 16-bit write. Removing this cast may corrupt SPI frames on certain compiler versions.
Available via	<code>spi.h</code>

5. Future Enhancements

- Interrupt-driven transfers: Replace `TXE/RXNE` polling with `TXEIE/RXNEIE` interrupts and an ISR-managed ring buffer to free the CPU during data movement.
- DMA support: Integrate `DMA1` or `DMA2` streams for zero-copy bulk transfers, reducing CPU load in high-throughput applications.
- Multi-slave support: Parameterize the CS pin so that the driver can manage multiple GPIO lines, enabling communication with several SPI devices on the same bus.
- SPI Mode selection: Extend `spi_init()` with a mode parameter (`CPOL/CPHA` pair) to support Mode 0, 1, and 2 in addition to the current Mode 3.
- 16-bit data frame format: Add a `DFF` configuration option to support 16-bit transfers for display controllers and DACs that require wider frames.

- Circular DMA streaming: Support continuous sensor sampling by configuring DMA in circular mode, writing directly to a double buffer shared with the application.

6. References

- [1] STMicroelectronics. (2017). RM0383 Reference Manual: STM32F411xC/E Advanced ARM-based 32-bit MCUs. Rev 3.
https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STMicroelectronics. (2017). DS10314 Datasheet: STM32F411RE. Rev 7.
<https://www.st.com/resource/en/datasheet/stm32f411re.pdf>
- [3] STMicroelectronics. (2016). UM1724 User Manual: STM32 Nucleo-64 Boards. Rev 14.
https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf
- [4] Analog Devices. (2009). ADXL345 Datasheet: 3-Axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g Digital Accelerometer. Rev D.
<https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf>
- [5] IEEE. (2009). IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions. IEEE Computer Society.